**TRƯỜNG ĐẠI HỌC CÔNG NGHỆ THÔNG TIN – ĐHQG-HCM**
**KHOA KỸ THUẬT MÁY TÍNH**

## IT012 – TỔ CHỨC VÀ CẤU TRÚC MÁY TÍNH II

# LAB 02

## ỨNG DỤNG CỦA MẠCH SỐ

| **Họ và tên** | VŨ VĂN HUỲNH | **MSSV** | 25520756 |
|---|---|---|---|
| **Lớp** | IT012.R11.2 | **Ngày thực hành** | 12/10/2026 |
| **Tự chấm** | 9/10 | **Link git** | https://github.com/vannhuynh/IT012 |

## 1. Tổng quan buổi thực hành

Sinh viên dùng phần mềm **Logisim** để biến kiến thức Chương 4 (mạch số, mạch tổ hợp, mạch tuần tự, latch, flip-flop, thanh ghi) và Chương 5 (bộ cộng, ALU, bộ chọn, bộ so sánh, bộ giải mã, tập thanh ghi) thành một bài toán xuyên suốt: **tự xây một datapath 8 bit của CPU thu nhỏ**, từng khối một, theo chu trình **dự đoán – dựng mạch – quan sát – giải thích**.

Sau buổi thực hành, Sinh viên có thể:

- Thiết kế và dựng bộ cộng/trừ từ cổng luận lý theo quy trình 4 bước (bảng chân trị → biểu thức → tối ưu → sơ đồ), đọc được các **cờ** Z, N, C, V và phát hiện **tràn số**.
- Dựng **ALU 8 bit** từ bộ chọn (MUX), bộ cộng/trừ và cổng luận lý; dùng ALU để so sánh hai số và chỉ ra **giới hạn** của quy tắc so sánh bằng phép trừ.
- Phân biệt **latch** (tích cực theo mức) với **flip-flop** (tích cực theo cạnh) bằng dạng sóng; tự dựng D latch và ghép hai latch thành D flip-flop.
- Giải thích vì sao thanh ghi dùng flip-flop và vì sao vòng hồi tiếp "thanh ghi → ALU → thanh ghi" vẫn hoạt động đúng.
- Dựng **tập thanh ghi 4 × 8 bit**, ghép với ALU thành đường dữ liệu và điều khiển bằng tay để chạy một chương trình ngắn (C = A + B, hoán đổi hai thanh ghi).

| **Phần** | **Nội dung** | **Thời gian gợi ý** | **Điểm** |
|---|---|---|---|
| **Phần 0** | Làm quen bus nhiều bit, mạch con, clock; kiểm tra môi trường | 15 phút | |
| **TN 1** | Bộ cộng/trừ 4 bit | 45 phút | 2,0 |
| **TN 2** | ALU 8 bit và bộ so sánh | 45 phút | 1,5 |
| **TN 3** | Mạch tuần tự: D latch, D flip-flop | 45 phút | 3,0 |
| **TN 4** | Tập thanh ghi và đường dữ liệu 8 bit | 60 phút | 3,0 |
| **Kết** | Tổng kết, phản tư, nộp bài | 15 phút | 0,5 |
| **Tổng** | | **225 phút** | **10,0** |

**Quy ước làm bài**

- **Ô nền vàng nhạt** là chỗ Sinh viên điền hoặc trả lời trực tiếp vào file này (gõ chữ, hoặc dán ảnh chụp màn hình vào khung nét đứt).
- Ký hiệu: NOT là dấu phẩy trên (A'); AND viết liền hoặc A·B; OR là A + B; XOR là A ⊕ B. Số thập lục phân viết 0x3C. Với bus nhiều bit, A[7] là bit trọng số cao nhất (MSB). Số có dấu dùng biểu diễn **bù 2** (Chương 2).
- Mỗi thí nghiệm theo chu trình **DỰ ĐOÁN → QUAN SÁT → GIẢI THÍCH**. Ghi dự đoán **trước** khi mô phỏng; dự đoán sai **không bị trừ điểm**, phần giải thích vì sao sai mới được chấm.
- Các mạch con (subcircuit) tạo ở thí nghiệm trước được **dùng lại** ở thí nghiệm sau. Đặt tên đúng quy ước và không xoá.

## PHẦN 0 — Làm quen bus, mạch con và clock (15 phút)

Sinh viên đã dùng Logisim ở Lab 01; phần này chỉ bổ sung các kỹ năng mới cho mạch nhiều bit và mạch tuần tự. Giảng viên hướng dẫn nhanh theo tài liệu *Hướng dẫn sử dụng Logisim*. Sinh viên tự đánh dấu ☑ khi thực hiện được từng kỹ năng (bỏ qua nếu đã thành thạo).

| ☑ | Kỹ năng | Gợi ý |
|---|---|---|
| ☐ | Tạo project mới; đặt tên circuit đầu tiên là **Warmup**; lưu file **IT012_Lab02_<MSSV>.circ** | Mỗi thí nghiệm dùng các circuit riêng trong **cùng một file** (Project ▸ Add Circuit); tên circuit theo quy ước ở đầu mỗi thí nghiệm. |
| ☐ | Đặt Input Pin / Output Pin **nhiều bit** (thuộc tính Data Bits); đổi từng bit bằng Poke tool | Pin 8 bit hiển thị 8 ô bit. Dùng **Probe** (thư viện Wiring) và đổi thuộc tính **Radix** để đọc ở dạng nhị phân, thập lục phân, thập phân có dấu. |
| ☐ | Dùng **Splitter** để tách bus thành từng dây và ghép các dây thành bus | Fan Out = số dây, Bit Width In = độ rộng bus. Kiểm tra thứ tự bit (bit 0 là LSB). |
| ☐ | Tạo **mạch con**: dựng một circuit, rồi kéo circuit đó từ ngăn Explorer vào circuit khác như một linh kiện | Vị trí các pin trong circuit quyết định vị trí các chân của linh kiện; đặt nhãn pin rõ ràng. |
| ☐ | Dùng linh kiện có sẵn: **Plexers** (Multiplexer, Decoder), **Arithmetic** (Adder, Subtractor), **Memory** (D Flip-Flop, Register) | Chọn linh kiện rồi chỉnh Data Bits, Select Bits ở ngăn thuộc tính (góc dưới bên trái). |
| ☐ | Điều khiển mạch tuần tự: đổi tay một Input Pin **CLK**; dùng **Simulate ▸ Reset Simulation** để đưa mạch về trạng thái ban đầu | Cạnh lên xảy ra khi Poke CLK từ 0 lên 1. Với linh kiện Clock (Wiring), mỗi lần Simulate ▸ Tick Once đổi Clock một nửa chu kỳ. |
| ☐ | (Tuỳ chọn) Xem dạng sóng: **Chronogram** (Logisim-evolution) hoặc cửa sổ **Simulate ▸ Logging** (Logisim 2.7.x) | Tuỳ phiên bản; không bắt buộc, dùng để kiểm tra lại Bảng 10 ở TN 3. |
| ☐ | Chụp màn hình mạch và dán vào khung ảnh của file này | Chụp cả mạch lẫn nhãn ngõ vào/ngõ ra và giá trị đang hiển thị. |

| **Phiên bản Logisim đang dùng** | Logisim evolution 5.0 |
|---|---|

**Đọc màu dây và giá trị trong Logisim**

**Xanh lá sáng** = 1 · **Xanh lá đậm** = 0 · **Xanh dương** = chưa xác định / chưa được điều khiển · **Đỏ** = lỗi/xung đột · **Cam** = hai đầu dây lệch số bit.

Ở mạch **tuần tự**, ngay sau khi mở file hoặc Reset Simulation, giá trị đang lưu có thể **chưa xác định** và hiển thị là **x** (hoặc dây xanh dương) cho đến khi mạch được ghi lần đầu. Đó là hành vi bình thường, không phải lỗi; nếu phiên bản Logisim của Sinh viên hiển thị khác, hãy ghi lại điều quan sát được.

**Bước 0 – Kiểm tra nhanh.** Trong circuit **Warmup**, đặt hai Input Pin 8 bit **A**, **B**, một **Adder** (Arithmetic, Data Bits = 8), một Output Pin 8 bit **S** và một Probe. **Dự đoán** S (viết hex) bằng tay trước khi Poke, rồi đọc S ở dạng thập phân **không dấu** và **có dấu** (bù 2).

| A | B | S = A + B (hex) | S không dấu | S có dấu (bù 2) |
|---|---|---|---|---|
| 0x20 | 0x0F | 0x2F | 47 | 47 |
| 0x7F | 0x01 | 0x80 | 128 | -128 |
| 0xFF | 0x01 | 0x00 | 0 | 0 |
| 0x64 | 0x64 | 0xC8 | 200 | -56 |

*Circuit Warmup chỉ để làm quen, không nộp. Hãy để ý: cùng một dãy bit, hai cách đọc cho hai giá trị khác nhau; Thí nghiệm 1 sẽ trả lời "phần cứng báo cho phần mềm biết kết quả có đáng tin hay không" bằng cách nào.*

# THÍ NGHIỆM 1 — Bộ cộng/trừ 4 bit và các cờ trạng thái

Thời lượng gợi ý: 45 phút · Circuit: **FA, ADDSUB4** · Điểm: 2,0

> **Tình huống**
> Phép cộng là phép toán nền tảng của ALU: phép trừ, tăng/giảm và so sánh đều quy về bộ cộng. Sinh viên sẽ tự xây bộ cộng 4 bit từ cổng luận lý, biến nó thành bộ cộng/trừ, rồi tự thiết kế mạch **phát hiện tràn số** để biết khi nào kết quả 4 bit không còn đúng, thông tin mà CPU thật luôn cung cấp dưới dạng các cờ.

## A. Bộ cộng toàn phần (FA) từ cổng luận lý

**Bước 1.1 – Dự đoán.** Cộng ba bit A + B + Cin cho kết quả từ 0 đến 3, viết được bằng hai bit (Cout S). Điền hai cột "dự đoán" của Bảng 1 bằng phép cộng số học, **chưa** dựng mạch.

DỰNG MẠCH **Bước 1.2.** Trong circuit **FA** (circuit mới), đặt 3 Input Pin **A, B, Cin** và 2 Output Pin **S, Cout**. Dựng theo slide: S = A ⊕ B ⊕ Cin; Cout = A·B + Cin·(A ⊕ B), dùng chung cổng A ⊕ B cho cả hai ngõ ra.

QUAN SÁT Dùng Poke tool (hoặc Analyze Circuit ▸ Table) để điền hai cột "đo".

**Bảng 1.** Bộ cộng toàn phần FA

| A | B | Cin | S (dự đoán) | Cout (dự đoán) | S (đo) | Cout (đo) |
|---|---|---|---|---|---|---|
| 0 | 0 | 0 | 0 | 0 | 0 | 0 |
| 0 | 0 | 1 | 1 | 0 | 1 | 0 |
| 0 | 1 | 0 | 1 | 0 | 1 | 0 |
| 0 | 1 | 1 | 0 | 1 | 0 | 1 |
| 1 | 0 | 0 | 1 | 0 | 1 | 0 |
| 1 | 0 | 1 | 0 | 1 | 0 | 1 |
| 1 | 1 | 0 | 0 | 1 | 0 | 1 |
| 1 | 1 | 1 | 1 | 1 | 1 | 1 |

**Câu 1.1 (Phân tích).** Cột Cout chính là hàm "có từ 2 ngõ vào trở lên bằng 1" mà slide Chương 4 dùng làm ví dụ thiết kế. Viết SoP tối giản của Cout (gom bìa Karnaugh 3 biến nếu cần) và tính C(B). So sánh với dạng Cout = A·B + Cin·(A ⊕ B) khi cổng A ⊕ B đã có sẵn để tạo S: tổng số cổng của cả mạch FA ở mỗi cách là bao nhiêu?

*Trả lời: F(A, B, Cin) = A.B + A.Cin + B.Cin, C(B) = 9.*
*Tổng số cổng của cả mạch FA ở mỗi cách:*
*Cách dùng 1: tổng là 6 cổng (2 XOR, 3 AND, 1 OR)*
*Cách dùng 2: tổng là 5 cổng (1 XOR, 1 XOR dùng chung, 2 AND, 1 OR)*

**Câu 1.2 (Phân tích).** Đếm **số tầng cổng** (độ sâu) của mạch FA ở bước 1.2 trên hai đường đi: (a) từ **Cin** đến **Cout**; (b) từ **A hoặc B** đến **Cout**. Ghi cách đếm.

*Trả lời: (a) từ Cin đến Cout qua 2 tầng.*
*(b) từ A hoặc B đến Cout qua 3 tầng.*
*Cách đếm: đếm số lượng cổng logic nối tiếp nhau trên đường đi dài nhất từ ngõ vào đến ngõ ra Cout.*

## B. Ghép thành bộ cộng 4 bit nối tiếp

**DỰNG MẠCH** **Bước 1.3.** Trong circuit **ADDSUB4**: Input Pin **A** (4 bit), **B** (4 bit), **SUB** (1 bit); Output Pin **S** (4 bit), **C** (1 bit). **Lần này chưa dùng SUB:** nối 4 mạch con FA liên tiếp (Cout của tầng i vào Cin của tầng i + 1; Cin của tầng 0 nối hằng 0). Dùng Splitter để tách A, B và gộp S. Đặt Probe (hoặc Output Pin tạm) cho ba dây nhớ **C1, C2, C3** giữa các tầng.

**DỰ ĐOÁN** Với ba phép cộng ở Bảng 2, dự đoán S, các bit nhớ và **số tầng mà bit nhớ đã lan qua** (bắt đầu đếm từ khi C1 = 1).

**Bảng 2.** Bit nhớ lan truyền trong bộ cộng 4 bit (Cin = 0)

| A | B | S (nhị phân) | C1 | C2 | C3 | Cout | Số tầng bit nhớ lan qua |
|---|---|---|---|---|---|---|---|
| 0001 | 0001 | 0010 | 1 | 0 | 0 | 0 | 1 |
| 0111 | 0001 | 1000 | 1 | 1 | 1 | 0 | 3 |
| 1111 | 0001 | 0000 | 1 | 1 | 1 | 1 | 4 |

**Câu 1.3 (Đánh giá).** Với bộ cộng nối tiếp n bit, bit nhớ có thể phải đi qua tất cả các tầng. Dựa vào Câu 1.2, điền số tầng cổng của đường đi dài nhất (từ A[0]/B[0] đến Cout) ở Bảng 3 và rút ra công thức theo n. Nhận xét: vì sao CPU 32/64 bit thực tế không dùng thuần bộ cộng nối tiếp?

**Bảng 3.** Độ sâu của bộ cộng nối tiếp

| n (số bit) | Số tầng FA mà bit nhớ đi qua | Số tầng cổng đến Cout |
|---|---|---|
| 4 | 4 | 9 |
| 8 | 8 | 17 |
| 32 | 32 | 65 |
| n | n | 2n + 1 |

*Công thức, nhận xét: Với bộ cộng n bit, bit nhớ có thể phải đi qua tất cả các tầng công thức tính đường đi dài nhất từ A[0]/B[0] đến Cout là 2n + 1.*
*Nhận xét: trong bộ cộng nối tiếp, bit nhớ của tầng trước là cin của tầng sau. Tín hiệu nhớ lan truyền tuần tự qua tất cả các tầng. Với CPU 32/64 bit không dùng thuần bộ cộng nối tiếp vì độ trễ tăng tuyến tính, làm chu kỳ của clock quá dài.*

## C. Biến thành bộ cộng/trừ

**Câu 1.4 (Hiểu).** A − B có thể viết là A + (−B), mà theo Chương 2, −B = B' + 1 (bù 2). Từ đó giải thích vì sao chỉ cần thêm 4 cổng XOR (mỗi cổng nhận B[i] và SUB) và nối SUB vào Cin của tầng 0 là bộ cộng thành bộ cộng/trừ. Điều gì xảy ra ở hai trường hợp SUB = 0 và SUB = 1?

*Trả lời: Ta chỉ cần thêm 4 cổng XOR và nối SUB vào Cin của tầng 0. A – B = A + (-B) mà -B = B' + 1. Như vậy phép trừ thực ra vẫn là phép cộng. Bộ cộng đã có sẵn nên không cần sửa, ta chỉ cần điều khiển bit của B và dấu bằng 4 cổng XOR và SUB.*
*Khi SUB = 0, XOR giữ nguyên B và Cin của tầng 0 là 0, nên mạch tính A + B.*
*Khi SUB = 1, XOR đảo bit của B thành B' và Cin = 1 nên mạch tính A + B' + 1 = A – B.*

**DỰNG MẠCH** **Bước 1.4.** Thêm vào ADDSUB4: 4 cổng XOR cho B'[i] = B[i] ⊕ SUB; nối SUB vào Cin của tầng 0. Kiểm tra nhanh: SUB = 0, A = 0011, B = 0100 → S = 0111; SUB = 1, A = 0111, B = 0011 → S = 0100.

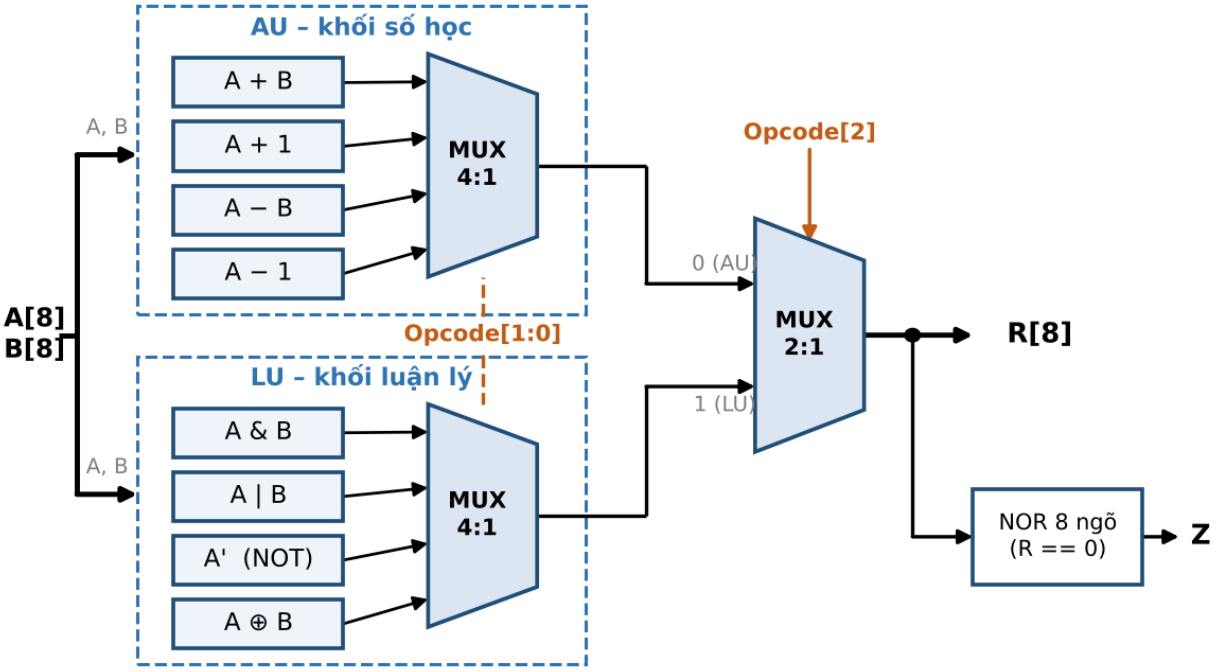
# THÍ NGHIỆM 2 — ALU 8 bit và bộ so sánh: quy tắc nào luôn đúng?

Thời lượng gợi ý: 45 phút · Circuit: **ALU8, LT** · Điểm: 1,5

**Tình huống**

Một CPU chỉ cần **một** khối tính toán dùng chung cho nhiều phép toán: ALU. Sinh viên dựng ALU 8 bit theo bảng Opcode ở slide Chương 5 (Hình 1), rồi dùng chính ALU để **so sánh hai số** thay vì thiết kế bộ so sánh riêng, vì bảng chân trị của bộ so sánh hai số 8 bit có 2^16 hàng. Slide nêu quy tắc: "Y = A − B; Y[MSB] = 1 ⇒ A < B; Y[MSB] = 0 ⇒ A > B". Sinh viên sẽ **kiểm tra** quy tắc này có luôn đúng không.

## A. Dựng ALU8 theo cấu trúc AU – LU – MUX




**Hình 1.** *Cấu trúc ALU 8 bit theo slide Chương 5. Opcode[1:0] chọn phép toán trong từng khối; Opcode[2] chọn khối AU hay LU.*

DỰNG MẠCH **Bước 2.1.** Trong circuit **ALU8**: Input Pin **A, B** (8 bit), **OP** (3 bit); Output Pin **R** (8 bit), **Z**. (a) **AU:** hai Adder (A + B, A + 1) và hai Subtractor (A − B, A − 1); hằng 1 lấy từ Constant 8 bit; chọn kết quả bằng Multiplexer 4:1 (8 bit) với Select = OP[1:0] (dùng Splitter tách OP). (b) **LU:** các cổng AND, OR, NOT, XOR có Data Bits = 8; chọn bằng một Multiplexer 4:1 khác với Select = OP[1:0]. (c) Multiplexer 2:1 (8 bit) chọn giữa AU và LU theo OP[2]. (d) **Z** = NOR của 8 bit R (Splitter + cổng NOR 8 ngõ vào).

*Gợi ý: ngõ vào số k của Multiplexer được chọn khi Select = k. Hãy đối chiếu thứ tự nối với bảng Opcode ở slide (000: A + B, 001: A + 1, 010: A − B, 011: A − 1, 100: A & B, 101: A | B, 110: A', 111: A ⊕ B).*

DỰ ĐOÁN Điền cột "dự đoán" của Bảng 6 bằng tay (hex, 8 bit) **trước khi** Poke.

QUAN SÁT **Ca 1:** A = 0x3C, B = 0x0F. **Ca 2:** A = 0x00, B = 0x01. Điền cột "đo" và cờ Z của Ca 2.

**Bảng 6.** Kiểm thử ALU8 với 8 Opcode

| Opcode | Phép toán | Ca 1: R dự đoán | Ca 1: R đo | Ca 2: R dự đoán | Ca 2: R đo | Ca 2: Z |
|---|---|---|---|---|---|---|
| 000 | A + B | 0x4B | 0x4B | 0x01 | 0x01 | 0 |

| Opcode | Phép toán | Ca 1: R dự đoán | Ca 1: R đo | Ca 2: R dự đoán | Ca 2: R đo | Ca 2: Z |
|---|---|---|---|---|---|---|
| 001 | A + 1 | 0x3D | 0x3D | 0x01 | 0x01 | 0 |
| 010 | A − B | 0x2D | 0x2D | 0xFF | 0xFF | 0 |
| 011 | A − 1 | 0x3B | 0x3B | 0xFF | 0xFF | 0 |
| 100 | A & B | 0x0C | 0x0C | 0x00 | 0x00 | 1 |
| 101 | A \| B | 0x3F | 0x3F | 0x01 | 0x01 | 0 |
| 110 | A' | 0xC3 | 0xC3 | 0xFF | 0xFF | 0 |
| 111 | A ⊕ B | 0x33 | 0x33 | 0x01 | 0x01 | 0 |

**Câu 2.1 (Phân tích).** Ở Ca 2, phép nào cho R = 0xFF? Giá trị đó là 255 hay −1? ALU có "biết" điều đó không? Z = 1 ở phép nào, và Z = 1 có đồng nghĩa với "hai toán hạng bằng nhau" không?

*Trả lời: ở ca 2, R = 0xFF khi A-B, A-1 và A'. 0xFF là 255 không dấu hoặc -1 có dấu, ALU không biết điều đó vì ALU chỉ thao tác xử lý các chuỗi bit, Z = 1 ở phép A&B. Z = 1 không có nghĩa "hai toán hạng bằng nhau" trong mọi phép toán. Z = 1 chỉ báo hiệu R = 0, Z = 1 chỉ có nghĩa A = B khi thực hiện phép trừ A – B.*

**Câu 2.2 (Áp dụng).** ALU này không có phép "chép" (MOV: R = A). Đề xuất **hai** cách dùng đúng ALU này để có R = A (ghi Opcode và giá trị cần đặt cho B). Cách nào dùng được khi ta **chỉ chọn được thanh ghi** cho A và B, không đặt được hằng số? (Sẽ dùng ở Thí nghiệm 4.)

*Trả lời:*
*Cách 1: OP = 000 (A + B) với B = 0x00 (hoặc OP = 010 với B = 0)*
*Cách 2: OP = 101 (A | B) hoặc 100 (A & B) với B = A*

## B. So sánh hai số bằng ALU

**DỰ ĐOÁN** Với 6 cặp (A, B) ở Bảng 7, tính R = A − B trên **8 bit**, rồi điền cột "Quy tắc slide kết luận" (dùng <, =, >). Điền hai cột "Thực tế" theo hai cách hiểu: A, B là số **có dấu** (bù 2) và là số **không dấu**.

**QUAN SÁT** Đặt Opcode = 010 trên ALU8, ghi R, Z, R[7] của từng cặp. Cột **LT đo** để trống, sẽ điền sau Bước 2.3.

**Bảng 7.** So sánh bằng phép trừ

| A | B | R = A − B (đo) | Z | R[7] | Quy tắc slide kết luận | Thực tế (có dấu) | Thực tế (không dấu) | LT đo |
|---|---|---|---|---|---|---|---|---|
| 0x05 | 0x07 | 0xFE | 0 | 1 | A < B | A < B | A < B | 1 |
| 0x09 | 0x09 | 0x00 | 1 | 0 | A = B | A = B | A = B | 0 |
| 0x64 | 0x9C | 0xC8 | 0 | 1 | A < B | A > B | A < B | 0 |
| 0x9C | 0x64 | 0x38 | 0 | 0 | A > B | A < B | A > B | 1 |
| 0xC8 | 0x0A | 0xBE | 0 | 1 | A < B | A < B | A > B | 1 |
| 0x0A | 0xC8 | 0x42 | 0 | 0 | A > B | A > B | A < B | 0 |

**Câu 2.3 (Phân tích).** Tìm các dòng mà quy tắc slide cho kết luận **sai**. Với mỗi dòng sai, nêu nguyên nhân (gợi ý: phép trừ 8 bit có bị tràn không? A và B đang được hiểu là số có dấu hay không dấu?). Quy tắc ở slide chỉ đáng tin trong điều kiện nào?

*Trả lời: các quy tắc slide cho kết quả sai khi so sánh số có dấu: dòng 3 (0x64 – 0x9C) và dòng 4 (0x9c – 0x64). Nguyên nhân là phép trừ tràn số: 100 – (-100) = 200 > 127 và - 100 – 100 = -200 < -128 nên R[7] sai dấu. Dòng 5 và 6, khi hiểu là số không dấu. Dòng 5 200 > 10 nhưng quy tắc cho A < B. Dòng 6 ngược lại, MSB không phải bit dấu nên quy tắc không áp dụng được.*

THIẾT KẾ **Bước 2.2 – Sửa quy tắc cho số có dấu.** Thiết kế mạch **LT** (A < B, số có dấu) chỉ dựa vào ba bit A[7], B[7], R[7] (R = A − B). Gợi ý: nếu A và B cùng dấu thì phép trừ không bao giờ tràn nên LT = R[7]; nếu khác dấu thì LT phụ thuộc dấu của A. Điền Bảng 8, bìa Karnaugh, rồi viết biểu thức tối giản.

**Bảng 8.** Bảng chân trị của LT(A7, B7, R7)

| A7 | B7 | R7 | LT (A < B có dấu) |
|---|---|---|---|
| 0 | 0 | 0 | 0 |
| 0 | 0 | 1 | 1 |
| 0 | 1 | 0 | 0 |
| 0 | 1 | 1 | 0 |
| 1 | 0 | 0 | 1 |
| 1 | 0 | 1 | 1 |
| 1 | 1 | 0 | 0 |
| 1 | 1 | 1 | 1 |

| A7 \ B7 R7 | 00 | 01 | 11 | 10 |
|---|---|---|---|---|
| **0** | 0 | 1 | 0 | 0 |
| **1** | 1 | 1 | 1 | 0 |

| | |
|---|---|
| LT tối giản = … | B7’R7 + A7B7’ + A7R7 |
| Số cổng cần dùng (kể cả cổng NOT) | 5 |

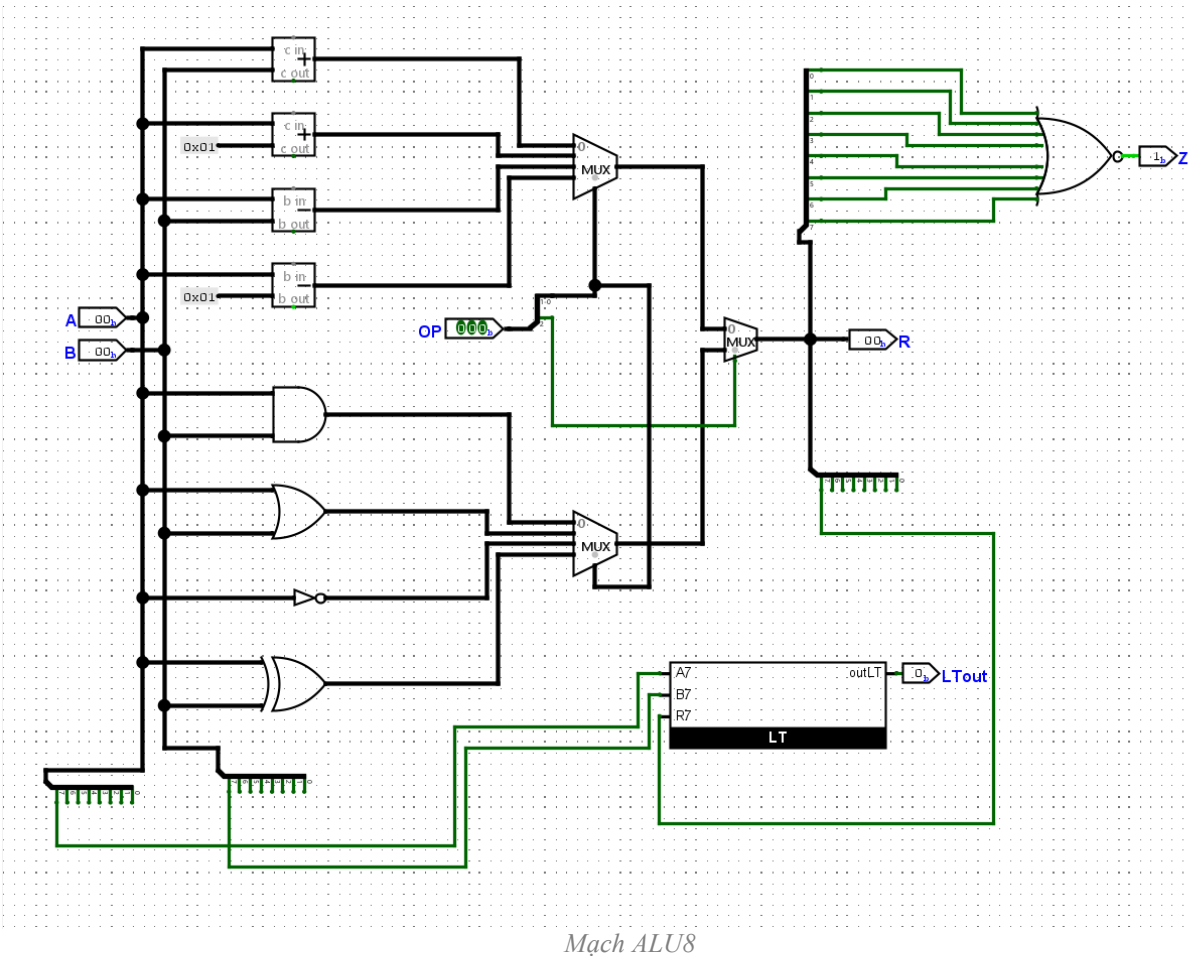
**Câu 2.4 (Phân tích).** Biểu thức LT quen thuộc không? (Gợi ý: so sánh với hàm đa số ở slide Chương 4 và với Cout của FA ở Thí nghiệm 1, trong đó một ngõ vào được lấy bù.) Giải thích bằng lời vì sao có sự giống nhau đó (nhớ lại A − B = A + B' + 1).

*Trả lời: LT cùng dạng với Cout của FA và ví dụ từ 2 ngõ vào trở lên bằng 1 ở Chương 4, lý do:*
*Vì A – B = A + B’ + 1, bit B ở tầng MSB đi vào bộ cộng dưới dạng B7’.*
*Nếu A7 = B7’(khác dấu) thì LT = A7. Nếu A7 khác B7’ (cùng dấu) thì LT = R7.*

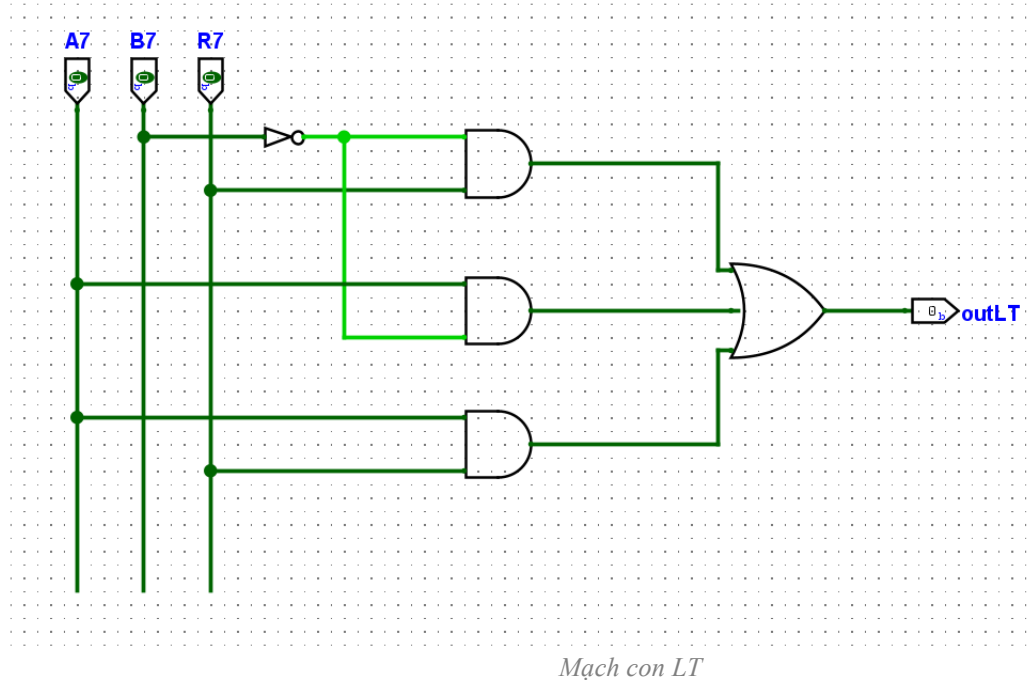
**DỰNG MẠCH** **Bước 2.3.** Dựng mạch con **LT** (ngõ vào A7, B7, R7; ngõ ra LT), nối vào ALU8 (A[7], B[7], R[7]) và điền cột **LT đo** của Bảng 7. LT có sửa được các dòng sai ở Câu 2.3 không?

**Câu 2.5 (Đánh giá – mở rộng).** Với số **không dấu**, A < B khi nào? (Gợi ý: ý nghĩa của cờ C khi trừ ở Câu 1.6b.) Kiểm tra điều kiện của Sinh viên trên hai dòng 0xC8 và 0x0A của Bảng 7 bằng cách tính A − B trên 9 bit. Sau đó, so sánh chi phí: bộ so sánh trực tiếp hai số 8 bit cần bảng chân trị bao nhiêu hàng, so với "ALU + LT"? Khi nào vẫn nên dùng bộ so sánh riêng?

*Trả lời: số không dấu: A < B ⇔ C = 0 khi tính A + B' + 1, tức không có carry-out.*
*Dòng 0xC8 – 0x0A: 0xC8 + 0xF5 + 1 = 0x1BE, carry = 1, nên A > B. Đúng.*
*Dòng 0x0A – 0xC8: 0x0A + 0x37 + 1 = 0x042, carry = 0, nên A < B. Đúng.*
*Chi phí: bộ so sánh trực tiếp cần bảng chân trị 2^16. ALU + LT chỉ cần thêm mạch LT 5 cổng trên ALU có sẵn.*
*Vẫn nên dùng bộ so sánh riêng khi cần tốc độ cao, khi cần so sánh song song nhiều cặp.*




*Mạch ALU8*



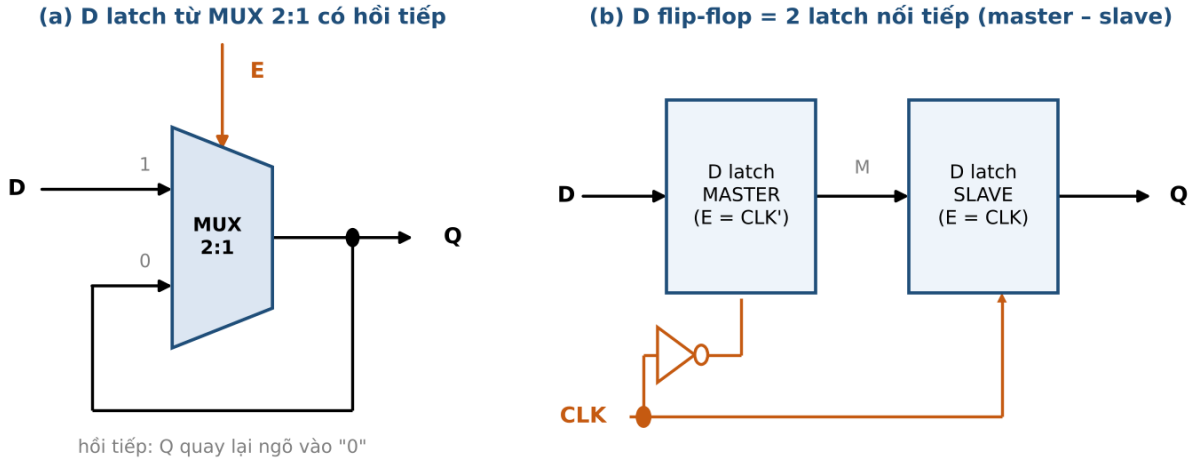

*Mạch con LT*

# THÍ NGHIỆM 3 — Mạch tuần tự: D latch và D flip-flop

Thời lượng gợi ý: 45 phút · Circuit: **TN3 (mạch con: DLATCH, DFF_MS)** · Điểm: 3,0

### Tình huống

ALU ở Thí nghiệm 2 là mạch tổ hợp: ngõ ra đổi ngay khi ngõ vào đổi và **không nhớ** gì. Muốn CPU giữ lại kết quả để dùng cho lệnh sau, cần **thiết bị lưu trữ** (slide Chương 4). Sinh viên sẽ tự dựng D latch từ bảng trạng thái ở slide, so sánh nó với D flip-flop có sẵn, rồi ghép hai latch thành một flip-flop và tìm hiểu vì sao thanh ghi phải dùng flip-flop.




**Hình 2.** *(a) D latch dựng từ MUX 2:1 có hồi tiếp; (b) D flip-flop ghép từ hai latch (master – slave).*

## A. D latch dựng từ bảng trạng thái

Slide Chương 4 mô tả D latch bằng bảng trạng thái kế tiếp: **Q+ = Q khi E = 0** (giữ), **Q+ = D khi E = 1** (nhận). Đó chính là một **MUX 2:1** mà ngõ vào "giữ" được nối với **ngõ ra Q của chính nó** (Hình 2a).

**DỰNG MẠCH** **Bước 3.1.** Trong circuit **DLATCH**: Input Pin **D, E**; Output Pin **Q**; một Multiplexer 1 bit (Select = E, ngõ vào 1 = D, ngõ vào 0 = **Q nối ngược về**).

**DỰ ĐOÁN** Chọn **Simulate ▸ Reset Simulation**, rồi thực hiện lần lượt các thao tác ở Bảng 9 (đổi E và D bằng Poke). Điền Q dự đoán **trước**; nếu Q chưa xác định thì ghi **x**.

**QUAN SÁT** Sau đó điền cột "Q đo".

**Bảng 9.** Chuỗi thao tác trên DLATCH

| Bước | E | D | Q dự đoán | Q đo |
|---|---|---|---|---|
| 1 | 0 | 0 | U | U |
| 2 | 0 | 1 | U | U |
| 3 | 1 | 1 | 1 | 1 |
| 4 | 0 | 1 | 1 | 1 |
| 5 | 0 | 0 | 1 | 1 |
| 6 | 1 | 0 | 0 | 0 |
| 7 | 1 | 1 | 1 | 1 |
| 8 | 1 | 0 | 0 | 0 |
| 9 | 0 | 0 | 0 | 0 |

| Bước | E | D | Q dự đoán | Q đo |
|---|---|---|---|---|
| 10 | 0 | 1 | 0 | 0 |

**Câu 3.1 (Hiểu).** Vì sao ở bước 1–2 Q chưa xác định dù đã có D? Điều đó nói gì về việc phải **khởi tạo** (reset) thiết bị lưu trữ trong mạch thật?

*Trả lời: khi E = 0, MUX chọn nhánh hồi tiếp Q. Nhưng Q chưa từng được ghi nên vẫn chưa xác định. Mạch thật khi cấp nguồn có Q ngẫu nhiên, nên phải khởi tạo bằng cách cho E = 1 một lần.*

**Câu 3.2 (Phân tích).** Slide Chương 4 định nghĩa mạch tổ hợp là mạch **không có hồi tiếp**; DLATCH có đường hồi tiếp từ Q về MUX. Khi E = 0, mô tả đường đi của tín hiệu để giải thích vì sao hồi tiếp làm mạch "nhớ". Khi E = 1 thì hồi tiếp còn tác dụng không?

*Trả lời:*
*Khi E = 0, MUX chọn ngõ vào 0 = Q, nên ngõ ra MUX (chính là Q) quay về làm ngõ vào của chính nó. Vòng kín này tự giữ giá trị cũ nên mạch có nhớ.*
*Khi E = 1, MUX chọn D, nhánh hồi tiếp bị chặn. Lúc này Q = D.*

## B. Latch và flip-flop: cùng ngõ vào, khác kết quả

**DỰNG MẠCH** **Bước 3.2.** Trong circuit **TN3**: Input Pin **D** và **CLK** (đổi bằng Poke, **không** dùng Clock tự chạy); một DLATCH với E = CLK (ngõ ra **Q1**); một **D Flip-Flop** (Memory, kích cạnh lên – Rising Edge) với ngõ vào D, CLK (ngõ ra **Q2**).

**DỰ ĐOÁN** Bảng 10 cho CLK và D theo từng **khe thời gian** (mỗi cột một khe; tại ranh giới giữa hai khe chỉ đổi **một** tín hiệu: CLK hoặc D, không bao giờ cả hai). Giả sử ban đầu Q1 = Q2 = 0. Điền hàng dự đoán Q1, Q2. Đây là bài "hoàn thành dạng sóng" ở slide Chương 4.

**QUAN SÁT** Đưa mạch về Q1 = Q2 = 0 (Reset Simulation; D = 0; CLK 0 → 1 → 0). Sau đó Poke từng khe theo bảng và ghi Q1, Q2 đo được. Hàng Q3 để trống đến Bước 3.3.

**Bảng 10.** Dạng sóng theo khe thời gian

| Khe | 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 10 | 11 | 12 | 13 | 14 | 15 |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| **CLK** | 0 | 0 | 1 | 1 | 0 | 0 | 1 | 1 | 0 | 0 | 1 | 1 | 0 | 0 | 1 | 1 |
| **D** | 0 | 1 | 1 | 0 | 0 | 1 | 1 | 1 | 1 | 0 | 0 | 1 | 1 | 0 | 0 | 0 |
| **Q1 dự đoán** | 0 | 0 | 1 | 0 | 0 | 0 | 1 | 1 | 1 | 1 | 0 | 1 | 1 | 1 | 0 | 0 |
| **Q1 đo** | 0 | 0 | 1 | 0 | 0 | 0 | 1 | 1 | 1 | 1 | 0 | 1 | 1 | 1 | 0 | 0 |
| **Q2 dự đoán** | 0 | 0 | 1 | 1 | 1 | 1 | 1 | 1 | 1 | 1 | 0 | 0 | 0 | 0 | 0 | 0 |
| **Q2 đo** | 0 | 0 | 1 | 1 | 1 | 1 | 1 | 1 | 1 | 1 | 0 | 0 | 0 | 0 | 0 | 0 |
| **Q3 đo (DFF_MS)** | 0 | 0 | 1 | 1 | 1 | 1 | 1 | 1 | 1 | 1 | 0 | 0 | 0 | 0 | 0 | 0 |

**Câu 3.3 (Phân tích).** Liệt kê các khe mà Q1 ≠ Q2. Với từng khe, giải thích bằng định nghĩa "tích cực theo **mức**" (latch) và "tích cực theo **cạnh**" (flip-flop) ở slide. Trong khoảng CLK = 1, D đổi một lần và Q1 đổi theo, còn Q2 thì không: vì sao?

*Trả lời: các khe mà Q1 khác Q2 là 3, 4, 5, 11, 12, 13.*
*Khe 3 và 11 (CLK = 1): latch tích cực theo mức, Q1 đổi theo D. Flip-flop chỉ lấy D tại cạnh lên nên Q2 giữ nguyên.*
*Khe 4-5 và 12-13(CLK = 0):latch đóng lại và giữ giá trị D cuối cùng lúc CLK = 1. Flip-flop giữ giá trị lấy mẫu ở cạnh lên.*
*Trong lúc CLK = 1, D đổi thì Q1 đổi theo vì latch đang nhớ. Q2 không đổi vì flip-flop đã lấy mẫu xong ở cạnh lên và bỏ qua D cho đến cạnh lên kế tiếp.*

## C. Vì sao thanh ghi dùng flip-flop mà không dùng latch?

**DỰNG MẠCH** **Bước 3.4.** Trong circuit TN3, dựng hai chuỗi hai tầng: **(i)** hai DLATCH nối tiếp (Q của latch 1 vào D của latch 2), cả hai có E = CLK; **(ii)** hai D Flip-Flop nối tiếp cùng CLK. Cả hai chuỗi dùng chung ngõ vào D.

**DỰ ĐOÁN** Ban đầu mọi Q = 0. Mỗi **xung** là CLK: 0 → 1 → 0, D giữ nguyên trong xung. Điền Q1, Q2 (viết dạng "Q1 Q2") sau mỗi xung, rồi đo.

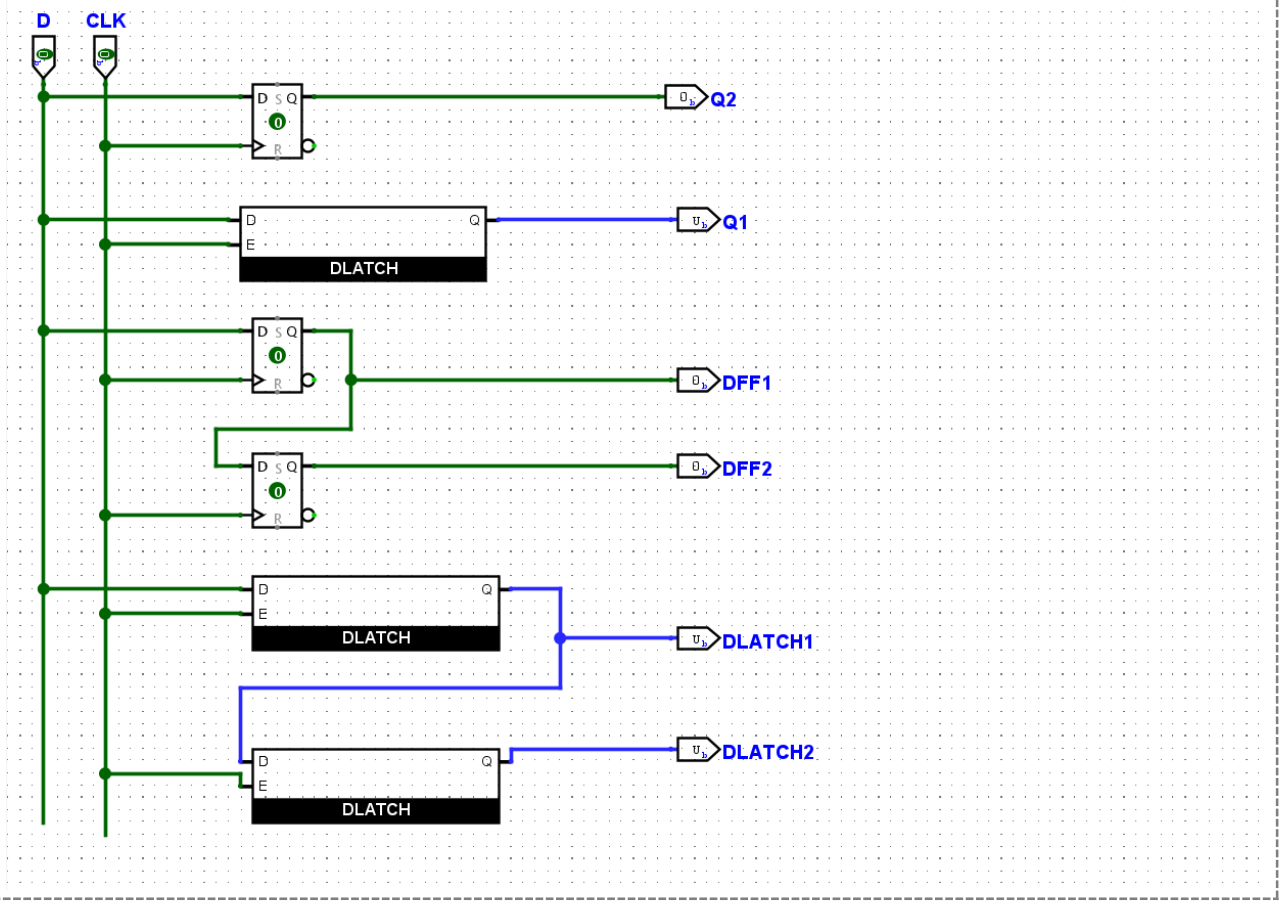
**Bảng 12.** Chuỗi hai tầng: latch và flip-flop

| Xung | D | 2 latch: dự đoán | 2 latch: đo | 2 flip-flop: dự đoán | 2 flip-flop: đo |
|---|---|---|---|---|---|
| 1 | 1 | 1 1 | 1 1 | 1 0 | 1 0 |
| 2 | 0 | 0 0 | 0 0 | 0 1 | 0 1 |
| 3 | 0 | 0 0 | 0 0 | 0 0 | 0 0 |

**Câu 3.5 (Đánh giá).** Từ Bảng 12, dữ liệu đi qua mấy tầng trong **một** xung ở mỗi cấu hình? Từ đó dự đoán điều gì xảy ra với lệnh **R1 ← R1 + R0** (ngõ ra ALU quay lại ngõ vào chính thanh ghi R1) khi R1 được làm bằng latch trong suốt lúc CLK = 1, và khi R1 làm bằng flip-flop. (Sinh viên sẽ kiểm chứng ở Thí nghiệm 4.)

*Trả lời: hai latch nối tiếp: dữ liệu đi qua 2 tầng trong 1 xung, vì cả hai cùng trong suốt khi CLK = 1.*
*Hai flip-flop nối tiếp: dữ liệu chỉ đi 1 tầng mỗi xung.*
*Với R1← R1 + R0 và R1 làm bằng latch: khi CLK = 1, ngõ ra R1 mới quay lại ALU và cộng R0 lần nữa, cứ thế lặp lại. R1 bị cộng nhiều lần, số lần không xác định, kết quả sai.*
*Với R1 làm bằng flip-flop: ALU tính R1 cũ cộng R0, và R1 chỉ nhận kết quả tại cạnh lên. Mỗi xung cộng đúng 1 lần, kết quả đúng.*

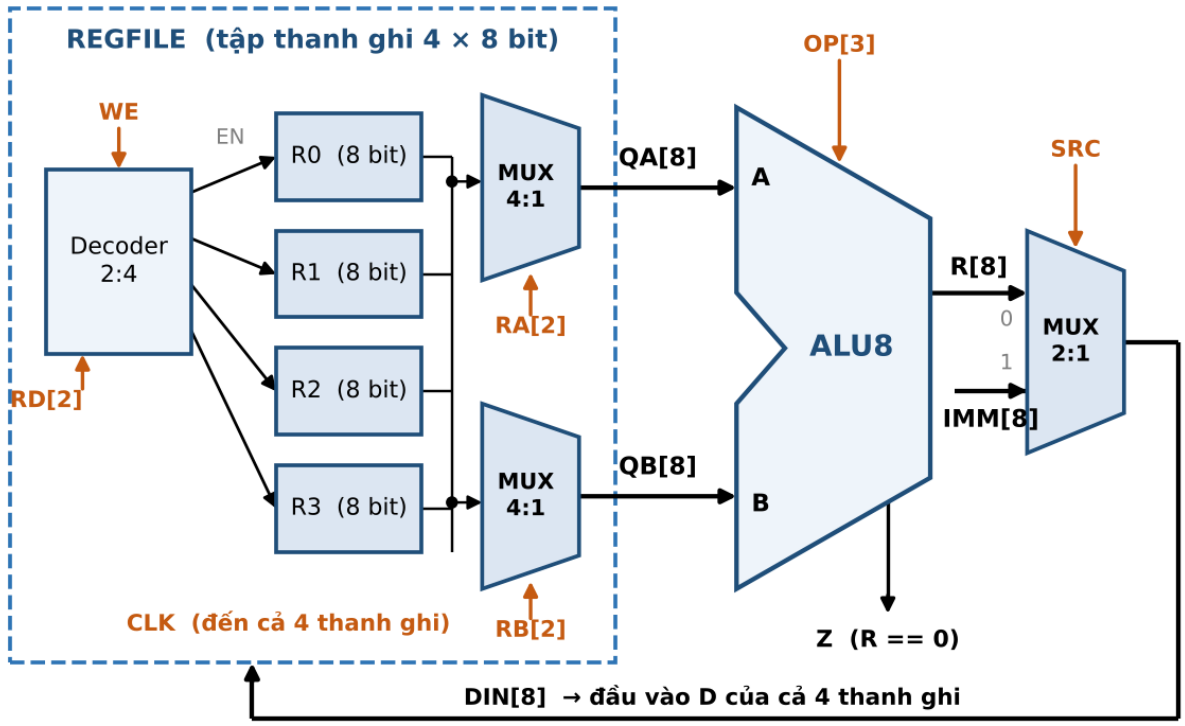
*Dán ảnh chụp circuit TN3 (Q1, Q2, Q3 và hai chuỗi hai tầng); nếu có, kèm ảnh Chronogram / Logging*



# THÍ NGHIỆM 4 — Tập thanh ghi và đường dữ liệu 8 bit

Thời lượng gợi ý: 60 phút · Circuit: **REGFILE, DATAPATH** · Điểm: 3,0

> **Tình huống**
> Slide Chương 5 minh hoạ lệnh **C = A + B**: đưa A, B từ bộ nhớ vào **tập thanh ghi**, **ALU** tính, kết quả ghi lại vào tập thanh ghi. Sinh viên ghép các khối đã dựng thành **đường dữ liệu** của một CPU thu nhỏ và điều khiển nó bằng tay: mỗi "lệnh" là một bộ tín hiệu điều khiển cộng với một xung CLK.




**Hình 3.** *Đường dữ liệu mini: REGFILE (decoder 2:4 + 4 thanh ghi + 2 MUX đọc) → ALU8 → MUX chọn nguồn ghi (SRC).*

## A. Tập thanh ghi 4 × 8 bit

Slide Chương 5 vẽ tập thanh ghi với một cổng đọc. ALU cần **hai toán hạng cùng lúc** nên REGFILE ở đây có **hai cổng đọc** (RA, RB) và **một cổng ghi** (RD).

DỰNG MẠCH **Bước 4.1.** Trong circuit **REGFILE**: Input Pin **CLK, WE** (1 bit), **RD** (2 bit, địa chỉ ghi), **DIN** (8 bit), **RA, RB** (2 bit, địa chỉ đọc); Output Pin **QA, QB** (8 bit) và **R0 … R3** (8 bit, để theo dõi). Dùng: một **Decoder 2:4** (Select = RD; nối WE vào ngõ Enable nếu có, hoặc dùng 4 cổng

AND với WE), **4 Register** 8 bit (D = DIN, En = ngõ ra thứ k của decoder, Clock = CLK) và **2 Multiplexer 4:1** 8 bit chọn Rk theo RA và theo RB.

**DỰ ĐOÁN** Với các bước 5–8 của Bảng 13, điền cột "R1 dự đoán" **trước** khi Poke. Mỗi bước kết thúc bằng một xung CLK (0 → 1 → 0) trừ khi ghi chú khác.

**QUAN SÁT** Ghi R0 … R3 (hex) sau từng bước.

**Bảng 13.** Ghi và đọc trên REGFILE

| Bước | Thao tác | R1 dự đoán | R0 | R1 | R2 | R3 |
|---|---|---|---|---|---|---|
| 0 | Vừa Reset Simulation (WE = 0) | 0x00 | 0x00 | 0x00 | 0x00 | 0x00 |
| 1 | RD = 0, WE = 1, DIN = 0x11, xung CLK | – | 0x11 | 0x00 | 0x00 | 0x00 |
| 2 | RD = 1, WE = 1, DIN = 0x22, xung CLK | – | 0x11 | 0x22 | 0x00 | 0x00 |
| 3 | RD = 2, WE = 1, DIN = 0x33, xung CLK | – | 0x11 | 0x22 | 0x33 | 0x00 |
| 4 | RD = 3, WE = 1, DIN = 0x44, xung CLK | – | 0x11 | 0x22 | 0x33 | 0x44 |
| 5 | RD = 1, **WE = 0**, DIN = 0x99, xung CLK | 0x22 | 0x11 | 0x22 | 0x33 | 0x44 |
| 6 | RD = 1, WE = 1, DIN = 0x99, **CLK vẫn = 0** | 0x22 | 0x11 | 0x22 | 0x33 | 0x44 |
| 7 | CLK: 0 → 1 (giữ CLK = 1) | 0x99 | 0x11 | 0x99 | 0x33 | 0x44 |
| 8 | CLK: 1 → 0 | 0x99 | 0x11 | 0x99 | 0x33 | 0x44 |

**Câu 4.1 (Hiểu).** Vì sao cần tín hiệu WE ngoài CLK? Điều gì xảy ra nếu bỏ WE (mỗi xung CLK đều ghi DIN vào thanh ghi do RD chọn) khi ta chỉ muốn **so sánh** hai thanh ghi mà không muốn ghi kết quả?

*Trả lời: cần tín hiệu WE để chủ động kiểm soát việc ghi dữ liệu vào thanh ghi.*
*Nếu bỏ WE, mọi xung CLK đều ghi dữ liệu DIN vào thanh ghi được chọn bởi RD. Khi đó, nếu ta thực hiện các phép toán chỉ cần kiểm tra/đọc dữ liệu, kết quả tính toán sẽ vô tình ghi đè làm mất dữ liệu gốc trong thanh ghi.*

**Câu 4.2 (Phân tích).** Khi RD = RA = 2, WE = 1 và DIN thay đổi: QA đổi **ngay khi DIN đổi** hay **chỉ sau cạnh lên**? Vì sao điều này quan trọng cho lệnh R2 ← R2 + R1 (liên hệ Bảng 12)?

*Trả lời: Khi RD = RA = WE = 1 , ngõ ra QA chỉ thay đổi sau cạnh lên của xung CLK.*
*Điều này đảm bảo an toàn cho lệnh $R_2 \leftarrow R_2 + R_1$: ALU lấy giá trị cũ của $R_2$ từ QA để tính tổng $R_2 + R_1$ sẵn sàng ở DIN. Khi cạnh lên CLK xuất hiện, kết quả mới mới được nạp vào $R_2$ đúng 1 lần.*

## B. Ghép đường dữ liệu

**DỰNG MẠCH** **Bước 4.2.** Trong circuit **DATAPATH**: dùng mạch con REGFILE, mạch con ALU8 và một Multiplexer 2:1 (8 bit) chọn DIN: **SRC = 0 → R** (kết quả ALU), **SRC = 1 → IMM** (hằng

số). Input Pin: **IMM** (8 bit), **SRC**, **OP** (3 bit), **RA, RB, RD** (2 bit), **WE**, **CLK**; Output Pin: **R0 … R3**, **R**, **Z**. Nối theo Hình 3 (A ← QA, B ← QB).

> **Quy ước "một lệnh"**
> - Một lệnh = đặt đúng các tín hiệu điều khiển (SRC, OP, RA, RB, RD, WE, IMM) **rồi** tạo một xung CLK (0 → 1 → 0).
> - Lệnh **nạp hằng**: SRC = 1. Lệnh **ALU**: SRC = 0. Lệnh **chỉ kiểm tra** (không ghi): WE = 0, đọc cờ Z.
> - Ô không dùng đến ghi dấu "–".

**DỰ ĐOÁN** Điền **từ vựng điều khiển** của 5 lệnh ở Bảng 14 (số thanh ghi viết dạng thập phân, OP dạng nhị phân, IMM dạng hex).

**Bảng 14.** Tín hiệu điều khiển của từng lệnh

| Lệnh | SRC | OP | RA | RB | RD | WE | IMM |
|---|---|---|---|---|---|---|---|
| (1) R0 ← 0x05 | 1 | | | | 0 | 1 | 0x05 |
| (2) R1 ← 0x07 | 1 | | | | 1 | 1 | 0x07 |
| (3) R2 ← R0 + R1 (C = A + B ở slide) | 0 | 000 | 0 | 1 | 2 | 1 | |
| (4) R3 ← R2 − R0 | 0 | 010 | 2 | 0 | 3 | 1 | |
| (5) So sánh R3 với R1 (không ghi) | 0 | 010 | 3 | 1 | | 0 | |

**QUAN SÁT** Chạy lần lượt 5 lệnh (từ trạng thái Reset Simulation) và ghi R0 … R3 (hex) và cờ Z sau mỗi lệnh.

**Bảng 15.** Kết quả chạy chương trình (1)–(5)

| Sau lệnh | R0 | R1 | R2 | R3 | Z |
|---|---|---|---|---|---|
| (1) | 0x05 | 0x00 | 0x00 | 0x00 | – |
| (2) | 0x05 | 0x07 | 0x00 | 0x00 | – |
| (3) | 0x05 | 0x07 | 0x0c | 0x00 | 0 |
| (4) | 0x05 | 0x07 | 0x0c | 0x07 | 0 |
| (5) | 0x05 | 0x07 | 0x0c | 0x07 | 1 |

**Câu 4.3 (Hiểu).** Lệnh (5) không ghi gì nhưng vẫn hữu ích. Z = 1 cho biết điều gì? So với quy tắc "MSB" ở Thí nghiệm 2, kết luận "bằng nhau" từ Z có luôn đúng không? Vì sao?

*Trả lời: Lệnh (5) không ghi dữ liệu nhưng cập nhật cờ trạng thái Z. $Z = 1$ chứng tỏ $R_3 - R_1 = 0 \Rightarrow R_3 = R_1$. Kết luận "bằng nhau" từ cờ $Z = 1$ luôn đúng trong mọi trường hợp (cho cả số có dấu và không dấu), vì $A = B \Leftrightarrow A - B = 0$, độc lập hoàn toàn với quy ước biểu diễn số.*

# TỔNG KẾT VÀ NỘP BÀI

## Danh sách kiểm tra trước khi nộp

| ☑ | Nội dung |
|---|---|
| ☐ | File **IT012_Lab02_<MSSV>.circ** có đủ các circuit **FA, ADDSUB4, ALU8, LT, DLATCH, DFF_MS, TN3, REGFILE, DATAPATH**; mọi Input/Output Pin đều có nhãn. |
| ☐ | File báo cáo này đã điền đầy đủ các bảng, câu hỏi và dán ảnh chụp mạch vào các khung nét đứt. |
| ☐ | Số liệu trong báo cáo khớp với mạch nộp; sau khi sửa mạch con (FA, ALU8…) đã kiểm tra lại các mạch dùng nó. |
| ☐ | Đặt tên file báo cáo: **IT012_Lab02_<MSSV>_BaoCao**; nộp cả hai file theo hướng dẫn của giảng viên. |

## Phản tư

**Câu R1.** Từ Thí nghiệm 1 và 2, Sinh viên rút ra điều gì về sự khác nhau giữa "kết quả **đúng về mặt bit**" và "kết quả **đúng về mặt ý nghĩa**" (có dấu / không dấu)? Phần cứng giúp phần mềm phân biệt hai điều đó bằng cách nào?

*Trả lời: Kết quả đúng về mặt bit" là việc mạch logic tính toán chính xác theo đúng các cổng Boolean. "Kết quả đúng về mặt ý nghĩa" đòi hỏi giá trị kết quả phải nằm trong phạm vi biểu diễn toán học của kiểu số tương ứng (có dấu / không dấu).*

**Câu R2.** Từ Thí nghiệm 3 và 4: trong DATAPATH, khối nào là mạch **tổ hợp**, khối nào là mạch **tuần tự**? Chỉ ra một vòng hồi tiếp (từ ngõ ra thanh ghi, qua ALU, quay về ngõ vào thanh ghi) và giải thích vì sao vòng này không gây hỗn loạn.

*Trả lời: Mạch tổ hợp: ALU8, MUX, Decoder 2:4, mạch LT.*
*Mạch tuần tự: Tập thanh ghi REGFILE (các D Flip-Flop).*
*Vòng hồi tiếp không gây hỗn loạn nhờ sử dụng D Flip-Flop kích theo cạnh (Edge-triggered). Cạnh lên clock chốt dữ liệu mới đúng 1 lần tại một thời điểm duy nhất, ngăn không cho dữ liệu quay lại lặp đi lặp lại trong chu kỳ.*

**Câu R3.** Nêu **một** lỗi hoặc nhầm lẫn Sinh viên đã gặp trong buổi thực hành, Sinh viên phát hiện ra bằng cách nào và bài học rút ra là gì?

*Trả lời: Khi đo Bảng 6, kết quả $A - 1$ ra 0x0E thay vì 0x3B, và $A \oplus B$ luôn ra 0x00. Em phát hiện bằng cách đối chiếu số đo với dự đoán tính tay: 0x0E đúng bằng $0x0F - 1$ nên mạch đang tính $B - 1$, còn XOR luôn bằng 0 gợi ý hai ngõ vào cùng một nguồn. Kiểm tra lại dây thì thấy khối trừ 1 nối nhầm vào bus B, và hai ngõ vào XOR cùng nối vào bus A. Sau khi sửa dây, hai phép cho 0x3B và 0x33 đúng như dự đoán.*
*Bài học: phải dự đoán trước khi đo, và khi số đo lệch thì suy ngược ra lỗi nối dây từ chính giá trị sai. Sau khi sửa mạch con, phải kiểm tra lại các mạch dùng nó.*

**Câu R4 (mở, không chấm).** Để DATAPATH chạy được một chương trình có vòng lặp hay rẽ nhánh ("nếu Z = 1 thì làm A, ngược lại làm B") mà **không cần người đặt tín hiệu bằng tay**, Sinh viên cần thêm những khối nào? Ghi ý tưởng, chưa cần vẽ mạch.

*Trả lời: Để DATAPATH tự động thực thi chương trình có rẽ nhánh/vòng lặp, cần bổ sung:*
- *Thanh ghi đếm chương trình: Lưu địa chỉ lệnh hiện tại.*
- *Bộ nhớ lệnh: Lưu trữ tập lệnh mã máy của chương trình.*
- *Bộ điều khiển : Giải mã lệnh để phát tự động các tín hiệu điều khiển .*
- *Mạch tính địa chỉ nhảy: Nhận cờ Z từ ALU để quyết định nạp $PC + 1$ hay địa chỉ nhảy $PC + offset$ vào PC.*

## Tài liệu tham khảo

- Slide bài giảng Chương 4 – Mạch số và Chương 5 – Ứng dụng mạch số (IT012, Khoa Kỹ thuật Máy tính, UIT).
- Slide bài giảng Chương 2 – Biểu diễn thông tin trong máy tính (số có dấu bù 2) và Chương 3 – Đại số Boole (bìa Karnaugh, chi phí C(B)).
- Tài liệu *Hướng dẫn sử dụng Logisim* do giảng viên cung cấp.